Pokemon Presentation

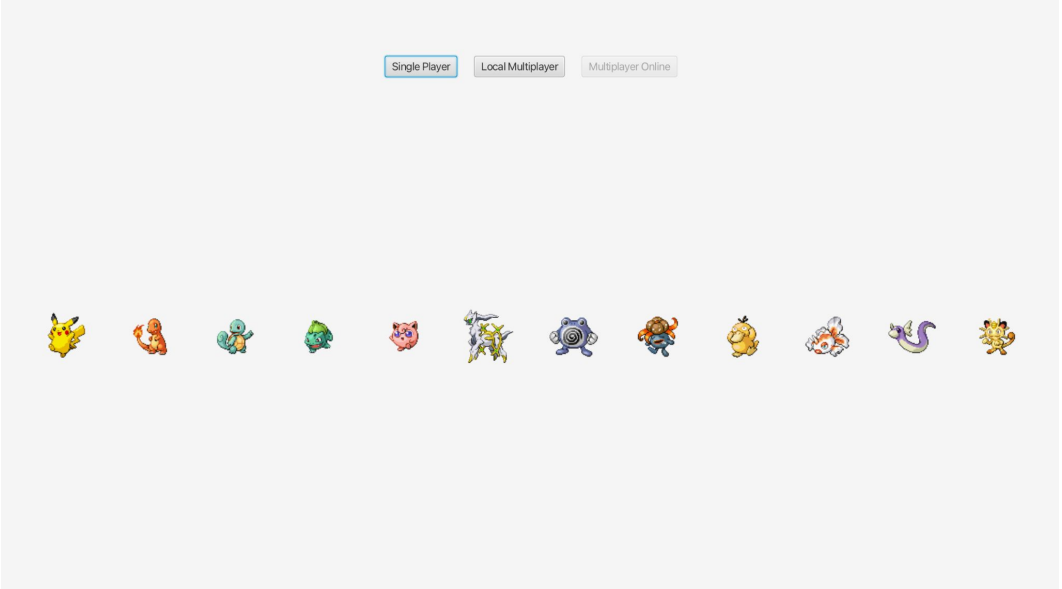
By Mohammed Alnasser, Jackson Jue, Conor Janesky, and Safwan Asif

Starting Menu

This is where the user first lands when opening up our pokemon game.

Here, the user can select the three pokemon that they wish to have on their team.

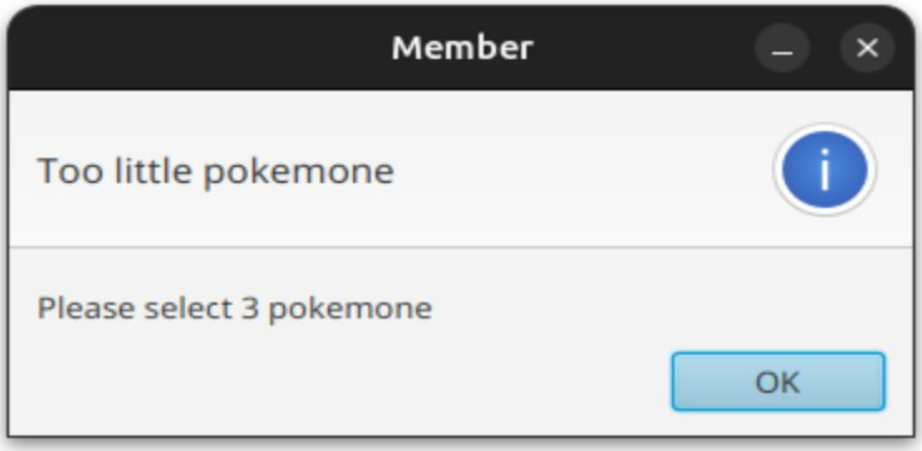
They are also presented with three options of Single Player, Local Multiplayer, and Multiplayer Online



Cases that we check for

If the user picks an incorrect quantity of pokemon, then it will show the alert to pick more or less in single player, local, and online multiplayer.

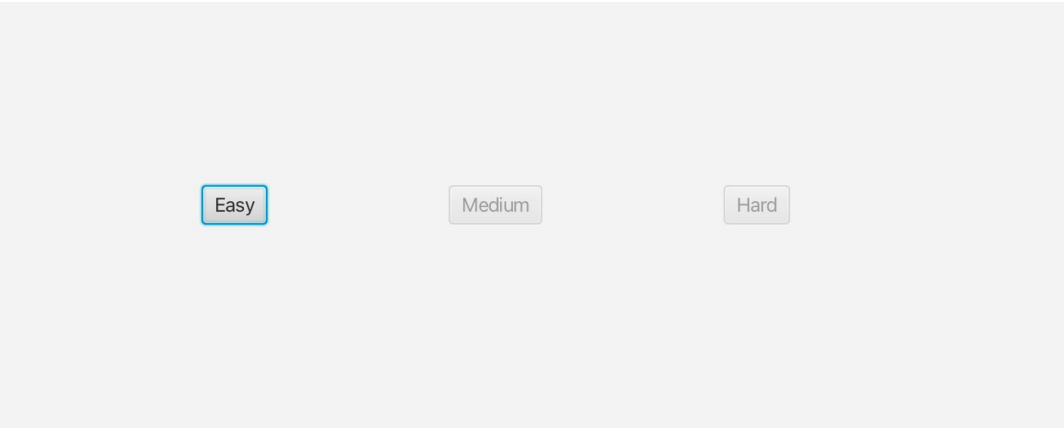
Additionally, we liked the idea of teams not repeating pokemon, so each team, in single player and multiplayer, do not allow the same pokemon creature on both teams.



Game Mode Selection

If the user picks Single player, they have an option to select either easy, medium, or hard for a dynamic gameplay experience and increase the difficulty.

Additionally, the user cannot progress to the harder difficulties until it they beat the current difficulty available to them. This is our progression mode.



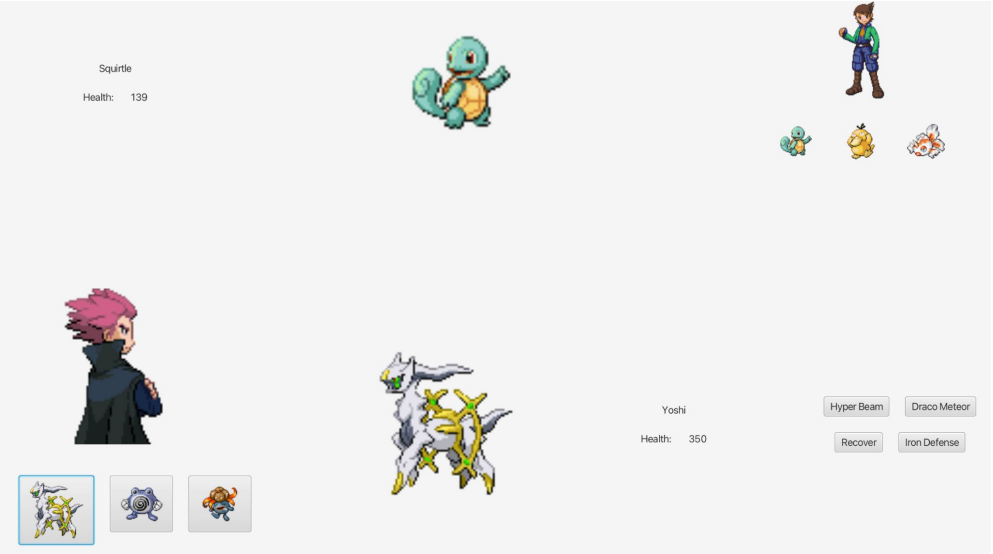
Battling!

User is able to start the battle here.

When a pokemon dies, it will automatically change to another (still alive) pokemon. The user is also presented with three other buttons under their trainer where they are allowed to swap with the current active creature.

Additionally it displays the current creatures name and its current health. This is the same for the opponent!

They also have 4 buttons on the right that correspond to the move of the active creature that they can press to start dealing damage.

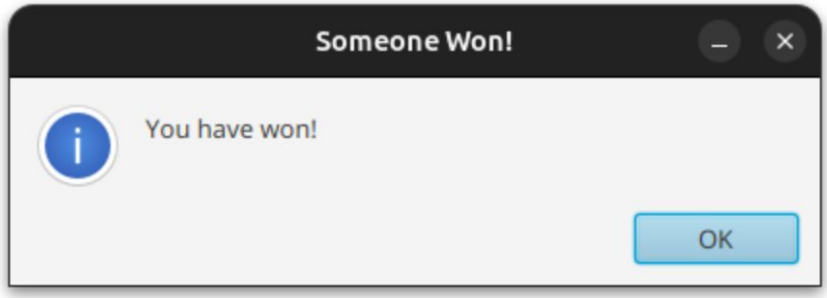


*also we had to name arceus ‘yoshi’ for copyright reasons :)

Ending Battle

These alerts pop up when one entire team has fainted!

The alert will pop up, and will inform the user of who won, and then once the user has selected “Okay” it will reset the game and send the user back to the main menu. Once in the main menu they can pick their team again and continue the fun! If the users are playing local multiplayer, the alert references the name that the use put in at the beginning of the battle.

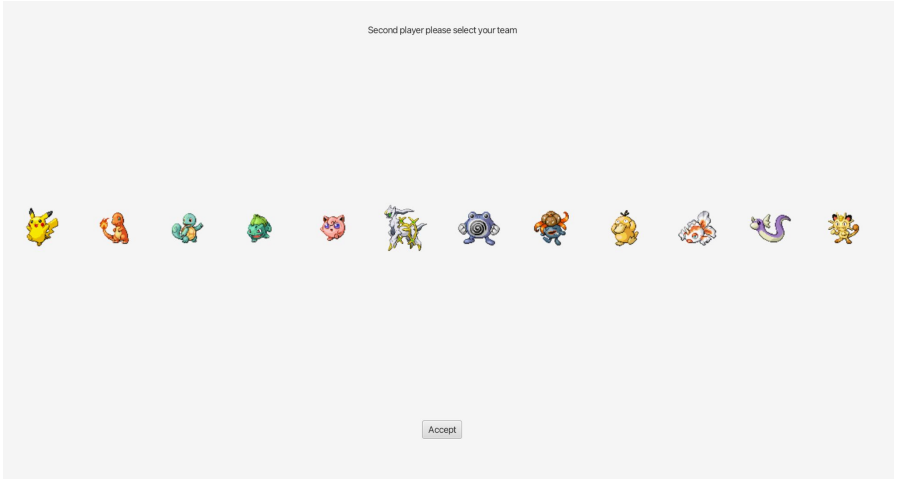


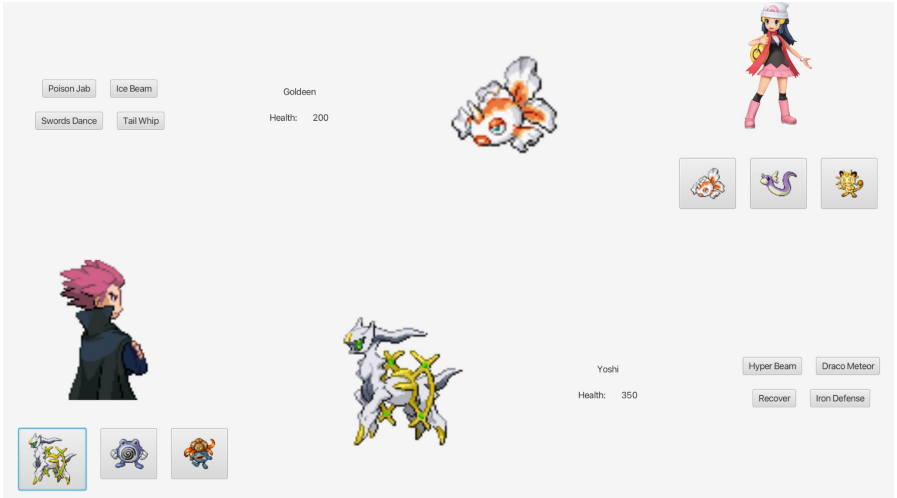


Local Multiplayer

Only after the User selects Local Multiplayer, the second player is able to choose their team and name (above).

Then, after going through the same checks as the first user, and the battle loads (below). Each player at the same pc is able to alternate their moves until one team is left standing.





WOW! Factors

We had multiple different ideas for the wow factors for this project. We decided to implement a typing system. This is so that each pokemon will deal either less effective or SUPER EFFECTIVE damage depending on who they are fighting.

Additionally, instead of creating only 3 GUIs, we made a unique GUI view for each of of the battles so that the player gets a interesting gameplay experience. We also had to include a GUI for when the second player would select their team for multiplayer local.

```
     */
    public enum Type {
        NORMAL, FIRE, WATER, GRASS, ELECTRIC, POISON, ICE, DRAGON, FAIRY
    }

    /**
     * this is the enum for the type of move (status)
     */
    public enum StatusEffect {
        HP_UP, ATTACK_UP, ATTACK_DOWN, DEFENSE_UP, DEFENSE_DOWN, NO_EFFECT
    }

    private String name;
    private Category moveCategory;
    private Type moveType;
    private StatusEffect effect;
    private int power;
    private int effectLevel;
```





Scrum Process

We had 3 different scrum meetings throughout the process of creating this project.

By the first scrum meeting, we had made a general game plan and split up who was going to do what for the basic logic for our game.

By the second meeting, we had been able to create a basic game flow and make the creatures fight each other.

In the final and third meeting, we discussed some trouble points and created a game plan to finish the necessary items for our project like networking and fleshing out the GUI and styling.

In each of these meetings we converted the notes taken into kanban items and added them to the ‘to-do’ section. We then moved each item into the ‘done’ category once it was complete.

We took additional notes that were taken in a discord that was shared by everybody, and screenshots of our notes can be seen in the github. They display our process from the beginning of the project to the end, updating and organizing each other surrounding the Sprint review meetings.

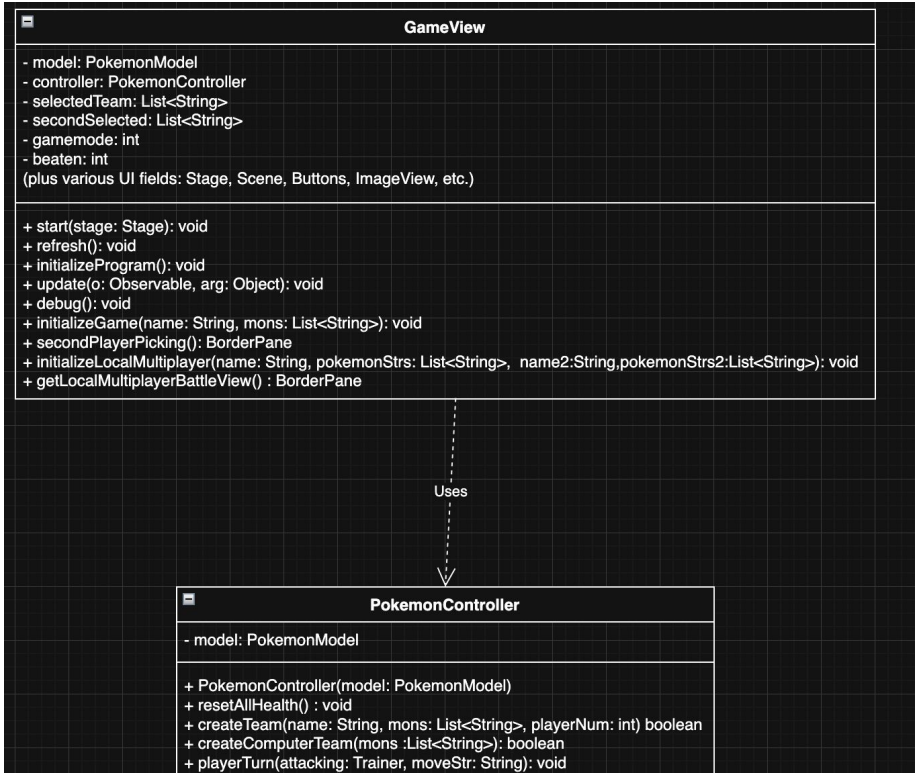
Additionally, we met throughout the week, and specifically on tuesdays we prepared for the scrum meeting by joining discord and discussing the actions points we would need to bring up. This meeting, along with the discussions before our sprint review, were our Sprint Planning and Sprint Retrospective meetings.

UML

We developed our UML on the first day of the project (when it was assigned) and used it and modified it throughout the process.

The uml helped a lot to determine the purpose of each class and how it fit into the MVC structure.

A full UML can be found in the SRC folder of our github and opened on draw.io



JUnit Test Cases

Additionally we wrote our JUnit test cases to ensure that every class had 100% coverage and was working appropriately and how we wanted them too.

We also avoided the GUI class and the Pokemon.java which only launches the GUI.

```java
package Tests;

import static org.junit.Assert.assertEquals;

class MoveTest {

    Move move;
    @Test
    void getMoves() {
        move = new Move("Thunderbolt", Category.PHYSICAL, Type.ELECTRIC, 60);
        assertEquals(move.getName(), "Thunderbolt");
        assertEquals(move.getMoveCategory(), Category.PHYSICAL);
        assertEquals(move.getMoveType(), Type.ELECTRIC);
        assertEquals(move.getPower(), 60);
    }

    @Test
    void test2() {
        move = new Move("Tail Whip", Category.STATUS, StatusEffect.DEFENSE_DOWN, 1);
        assertEquals(move.getName(), "Tail Whip");
        assertEquals(move.getMoveCategory(), Category.STATUS);
        assertEquals(move.getEffect(), StatusEffect.DEFENSE_DOWN);
        assertEquals(move.getEffectLevel(), 1);
    }
}
```